Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Information Systems Integration

Part 3 – Service Oriented Architecture and Web Services

Service Oriented Architecture

- Introduction

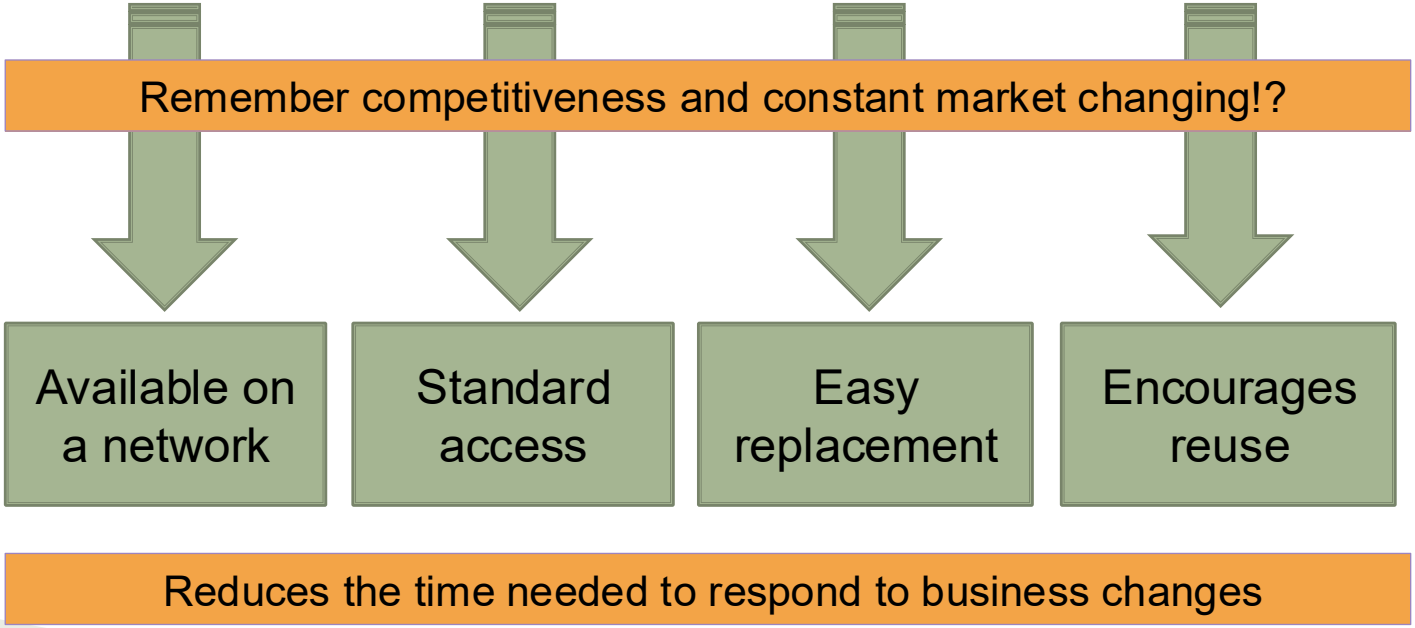
Background

- What is SOA (Service Oriented Architecture)
  - It is not a product
  - It is not an implementation
  - It is a concept!

Require technology and products at the infrastructure level

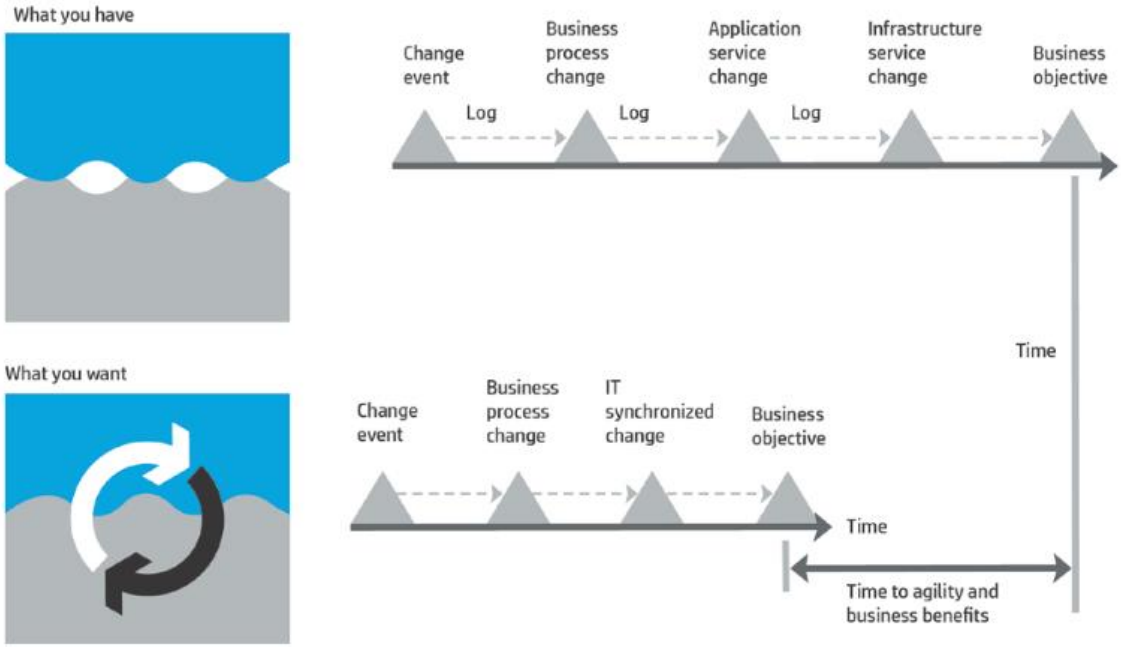
Background

- What is SOA (Service Oriented Architecture)
  - **Main idea:**
    - Loosely coupled components that support business process



Background

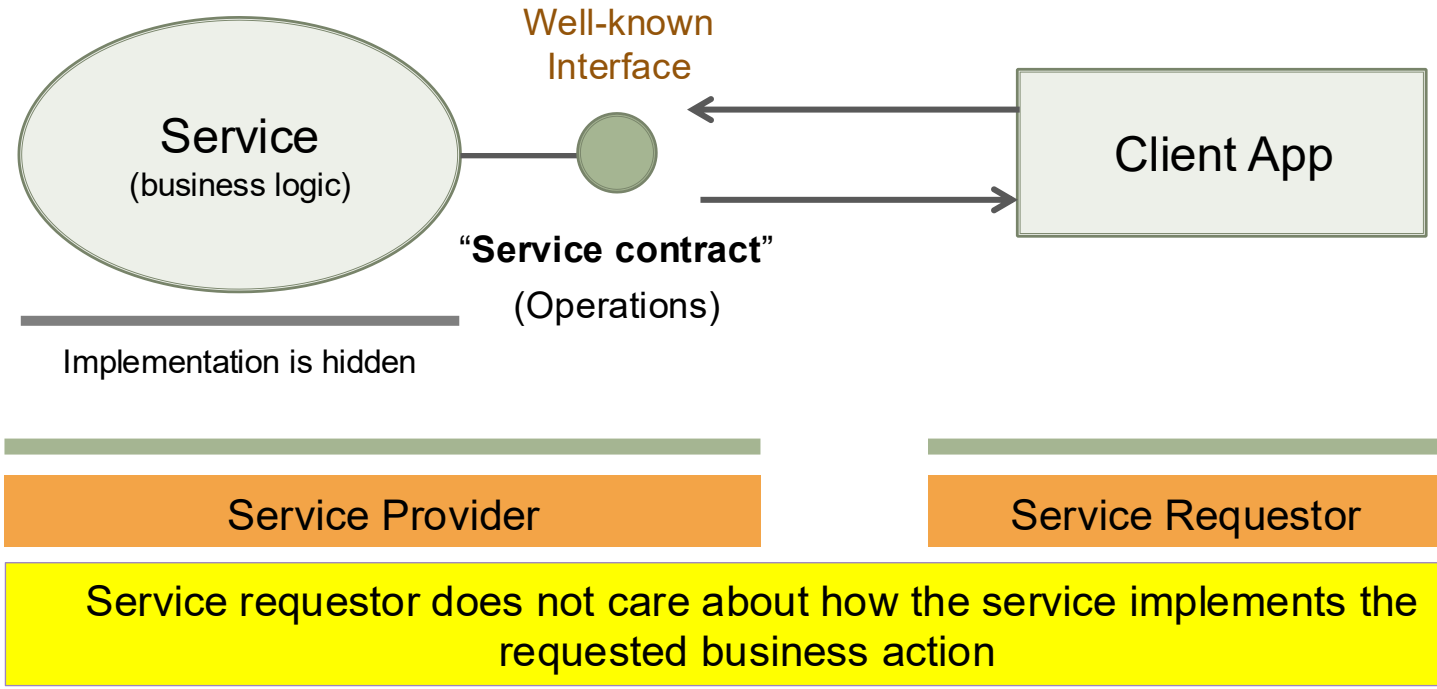
- The goal of SOA — business agility



[1]

Background

- SOA: but what is a service anyway?
  - A **basic** unit of business logic



Background

- SOA: principles behind service construction
  - 1. **Loose coupling**
    - Interdependency between services/parties?
  - 2. **Service abstraction**
    - Service must be completely **hidden** while interface is well-known
    - Black box approach

Background

- SOA: principles behind service construction
  - 3. **Service composition**
    - Ability to be part of another service
  - 4. **Service autonomy**
    - Control over resources it uses (performance and scalability)
  - 5. **Statelessness**
    - No need to save state information for longer periods of time (only during invocation)
    - High impact in scalability and performance (**clustering**!)

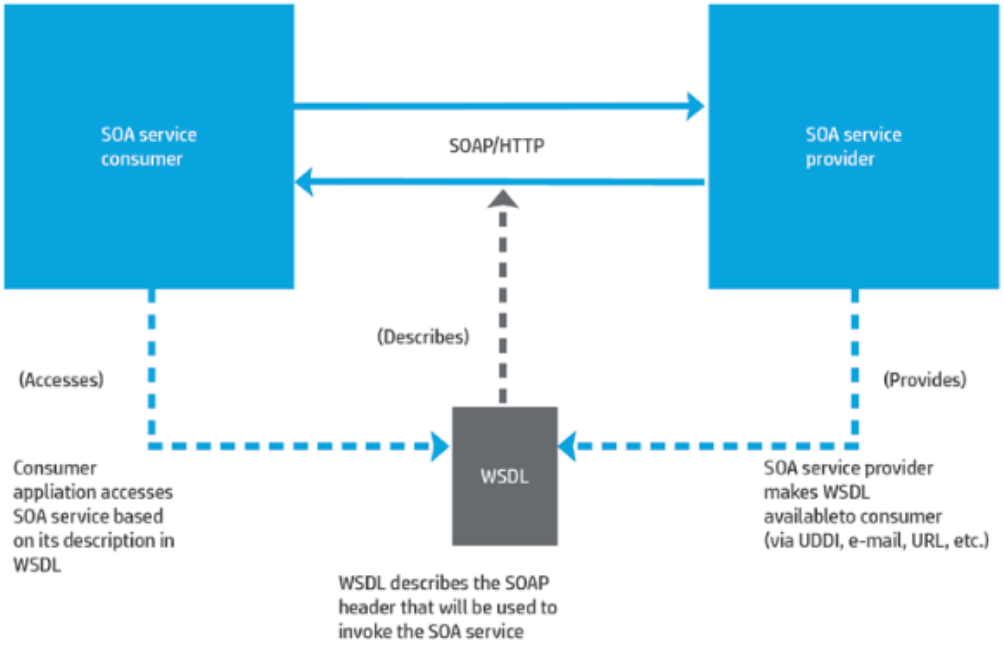
?

Background

- SOA: basic building blocks
  - **Interface description**
    - Which operations, returns and arguments, addressing…
    - Example: WSDL - Web Service Definition Language
  - **Service invocation**
    - The messaging protocol: ex. Simple Object Access Protocol (SOAP)
  - **Service discovery**
    - Publishing service interface : private or public registry

Background

- SOA: basic building blocks (Web Services oriented)

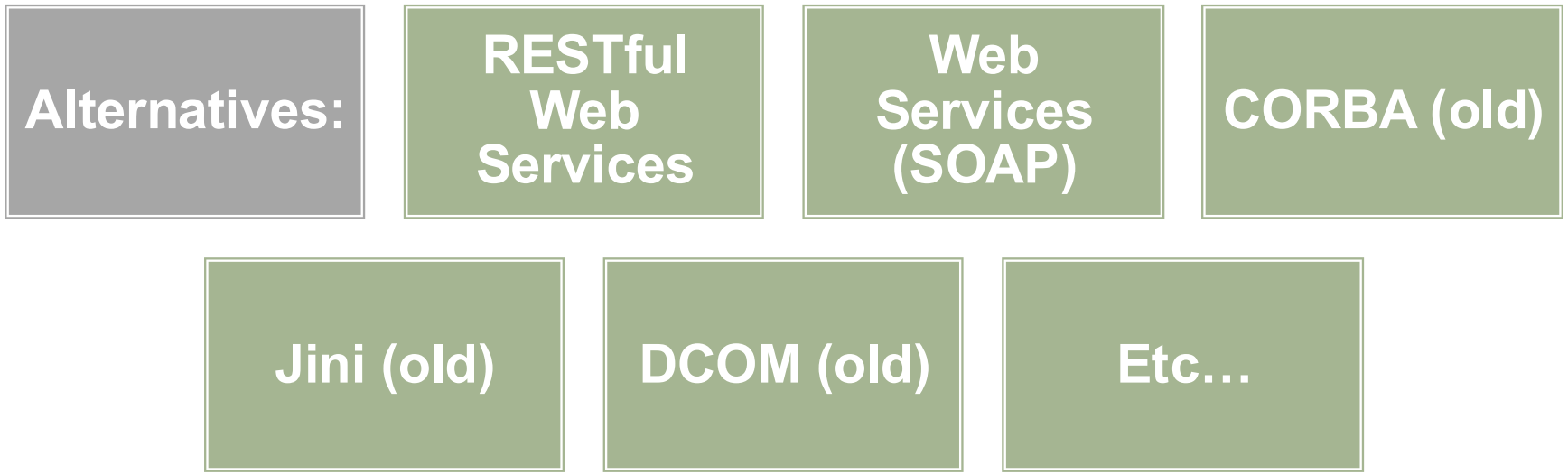


[1]

Web Services

- Introduction

Web Services are one of many materializations of SOA



Background

- What is a Web Service?
  - Web services = Web and XML for app-to-app communication
  - **What about traditional middleware (e.g., Jini, RMI,DCOM, RPC)?**
    - Traditional middleware does not support heterogeneity very well
    - Do not work across Internet
    - It is not so pervasive
    - Hard to maintain



Background

- **Advantages of Web Services**
  - Web Services: **platform and language independent**

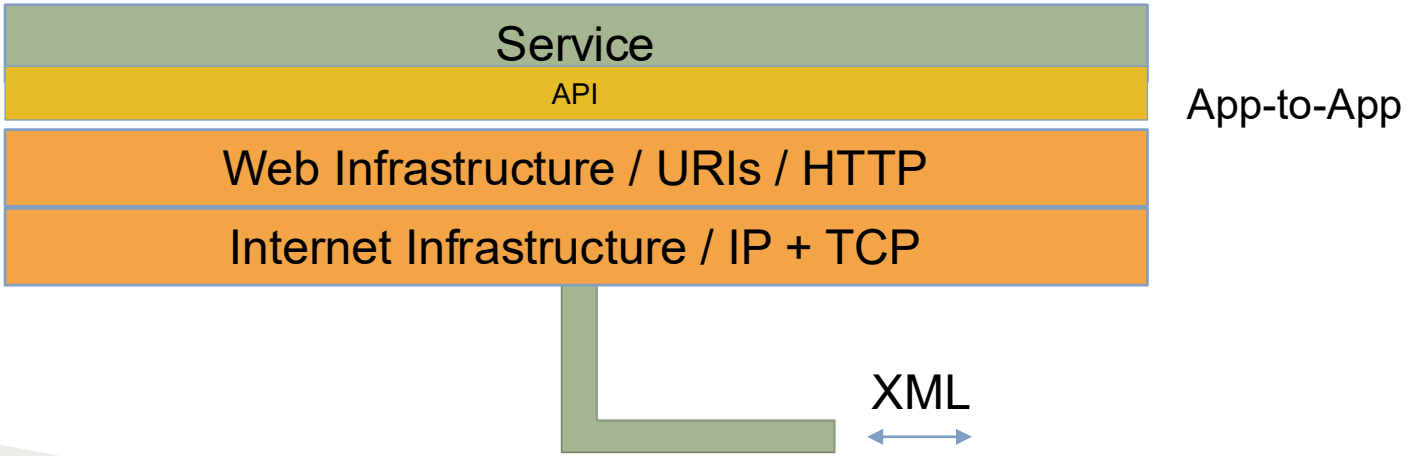
    It is the magic of Web and XML/JSON

  - Simplify the process of making apps talk to each other
    - Lower development costs
    - Faster time to market
    - Easier maintenance
    - Less tightly coupled than traditional middleware

Background

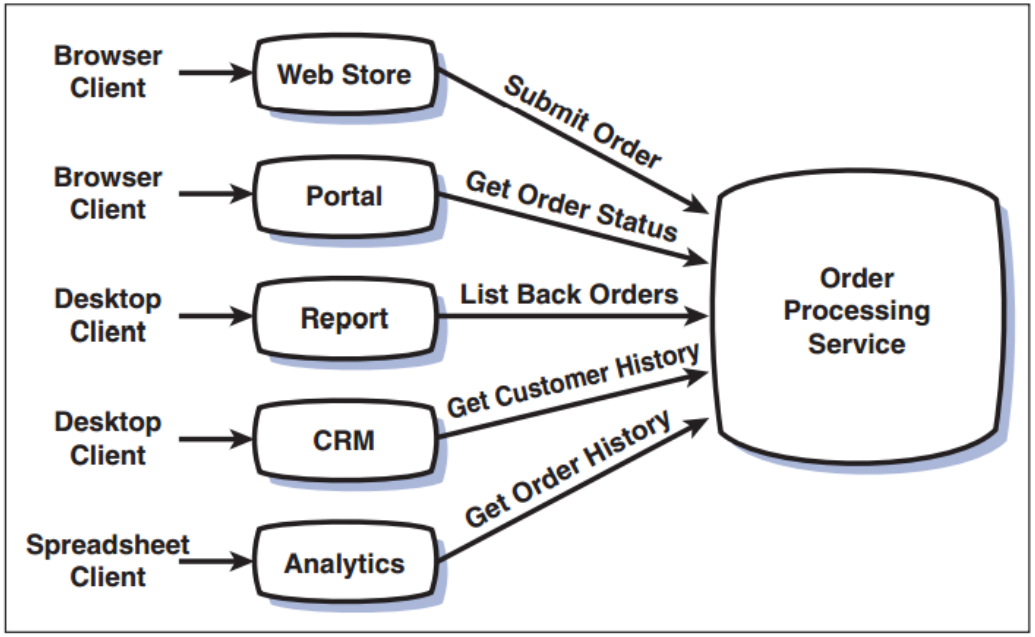
- What is a Web service (again)
  - Web Service = Web + Service

It's a service that lives in the Web
☺



Web Service Basics

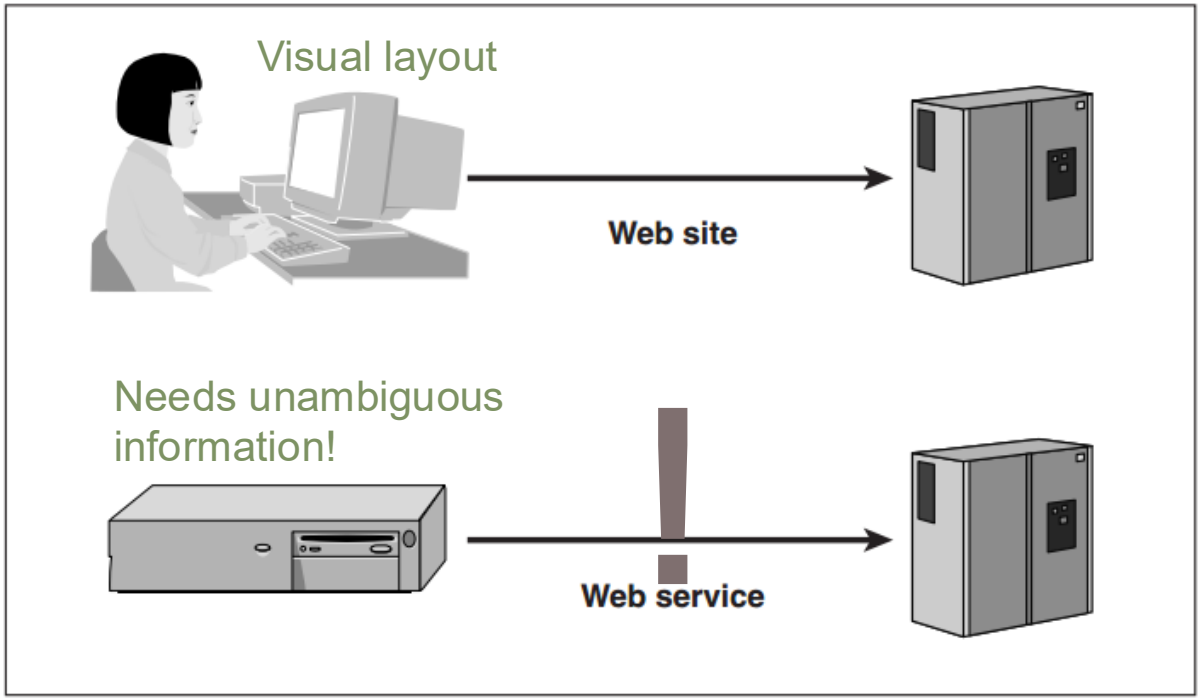
- Web services can be shared by different applications [David Chappell]



- Scalability!
- No duplicated work (implementation)
- Easier to manage and maintain

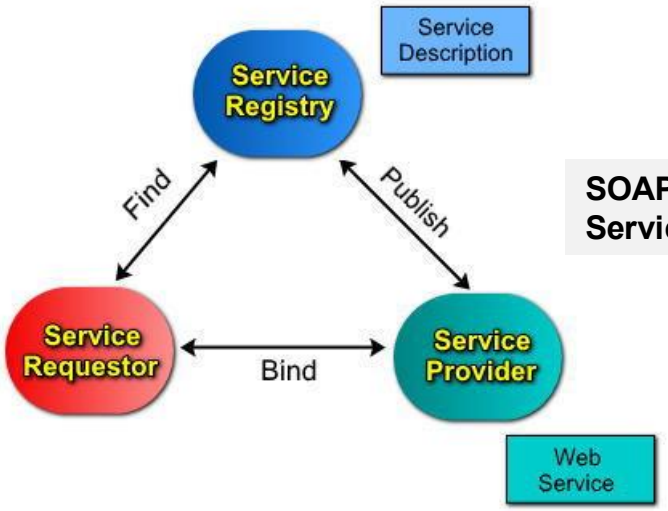
Web Service Basics

- Web services for machines and web pages for humans!



Web Service Basics

- Summary of characteristics
  - Web service is a (pervasive) Web resource
  - The interface is shown, implementation is hidden
  - Web services are registered in a **Web Service Registry** (eases discovery of a service to fit our needs)
  - Loosely coupled (XML based)



[IBM]

Web Service Basics

- **The business model**
  - (Usually) You do not sell a web service
  - You sell a product or other service types. Web services help to do that.

Web Service Basics

- The business model
  - **The Google example (search engine)**
    - Main interface is for humans/browser and is free
    - Business model: advertising
    - **Google also offers a searching API (for machines)**
      - Google Custom Search
        - No ads injected. Just goodwill!?

It is not a good example of business model, isn´t it? ☺

Web Service Basics

- The business model
  - **The FedEx example (FedEx QuickShip addon)**
    - Add-on for Outlook (uses company web services)
    - Calculates shipping rates, schedule pickups, etc.

Web service doesn't compete with the company's core business model. It enhances it!

Web Service Basics

- The business model
  - **The Amazon example (marketing affiliates Web Service)**
    - Amazon products are shown in the marketing affiliates web pages and users can search and buy from there!

This Web Service offers a Win-Win situation

Do not forget the **Amazon Web Services** (AWS) Cloud Computing Services

Web Service Basics

- The business model
  - **Summary**
    - Usually, Web Services support existing business models. They do not define a business model *per si*.

Web Service Basics

- **Final notes about SOA** [2]
  - SOA is centralized architecture!



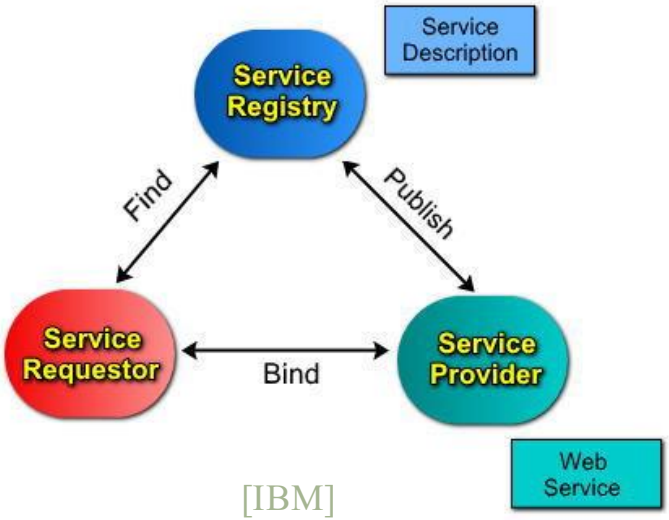
Web Service Basics

- **Final notes about SOA**
  - SOA is centralized architecture!
    - Bottleneck and SPOF
    - Some companies argue that SOA is too inflexible…

Alternatives?

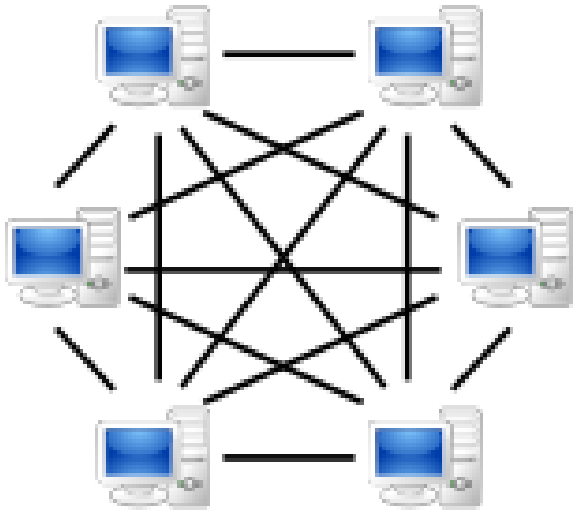
Web Service Basics

- **Final notes about SOA**
  - Peer-to-Peer model as an alternative (no roles)



[IBM]

SOA

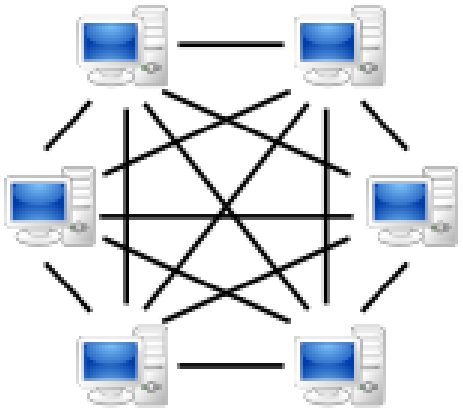


[Brian Duffy]

P2P

Web Service Basics

- **Final notes about SOA**
  - Peer-to-Peer model as an alternative (no roles)
    - No assigned roles (requestor, provider, registry)
    - Anyone could be a requestor, provider or registry

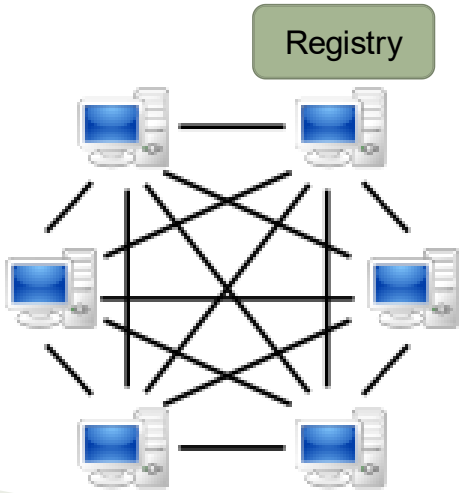


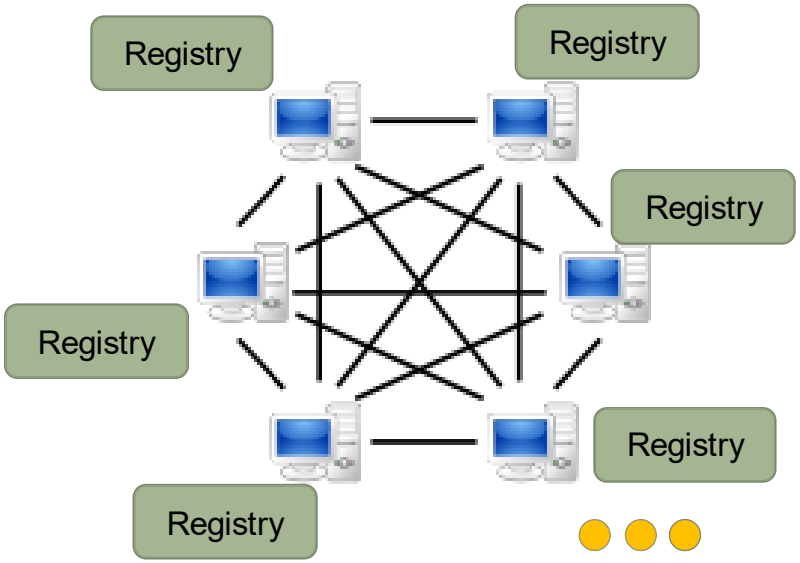
Examples: Napster, Gnutella, etc. remember?

Web Service Basics

- **Final notes about SOA**
  - Peer-to-Peer model as an alternative (no roles)

Napster	Gnutella
Centralized registry	Distributed registry





Web Service Basics

- **Final notes about SOA**
  - Advocates of P2P computing have often **failed to come** up with a business model that works in a decentralized environment…
    - This shouldn´t prevent us from looking to advantages of P2P model
      - No congestions
      - No bottlenecks
      - Resiliency (fault tolerant)
      - Greater availability

Web Service Basics

- **Final notes about SOA**
  - 2 ‘promising’ P2P frameworks:
    - JXTA (https://jxta.kenai.com)
    - BEEP (http://www.beepcore.org)

freedom.js is a js framework for building peer-to-peer (P2P) web apps.

“.NET P2P: Writing Peer-to-Peer Networked Apps with the Microsoft .NET Framework”

References

- [1] HP, “An introduction to SOA and the HP NonStop server environment”, Technical white paper

- [2] David A. Chappell and Tyler Jewell, “Java Web Services”, O’REILLY, 2002 [Chapter 2]